

BadSQL

Design Document

v0.1.0-alpha · C++23 · TCP Server Architecture

Autore: RickIsGone

Repository: github.com/RickIsGone/BadSQL

Linguaggio implementazione: C++23 + Boost | Build: CMake 3.30+

Stato: Alpha attiva · 2026

1. Overview del Progetto

BadSQL è un database server scritto da zero in C++23 con architettura client-server su TCP. Il server accetta connessioni sulla porta 5432 ed esegue query in un linguaggio SQL-like salvate in file con estensione .badsql. Il progetto costruisce incrementalmente un database engine completo partendo dai fondamentali: lexer, parser, executor e storage in-memory.

1.1 Stack Tecnologico

Componente	Scelta	Motivazione
Linguaggio	C++23	Performance, controllo memoria, std::expected, std::optional
Build system	CMake 3.30+	Cross-platform, standard de-facto
Dipendenze	Boost	ASIO networking, strutture dati, utilities
Protocollo rete	TCP porta 5432	Familiare (PostgreSQL), semplice da testare
Formato query	File .badsql	Estensione custom, sintassi SQL-like

1.2 Stato Implementativo (v0.1.0)

Feature	Stato	Descrizione
Lexer	✓ Implementato	Tokenizzazione con string_view
Token (enum + struct)	✓ Implementato	TokenType + Position tracking
Token matching	⚙ In corso	unordered_map con string_view
Parser	Pianificato	Costruzione AST da token stream
Executor	Pianificato	Interpretazione AST
TCP Server	⚙ In corso	Socket + recv/send
Storage in-memory	⚙ Parziale	RAM only, nessuna persistenza
WHERE clause	✗ Non iniziato	Filtro righe - prossima priorità
UPDATE / DELETE	✗ Non iniziato	Da implementare
Persistenza su disco	✗ Non iniziato	Fase v0.4
Indici	✗ Non iniziato	Fase v0.5
Transazioni	✗ Non iniziato	Fase v1.0

2. Architettura del Sistema

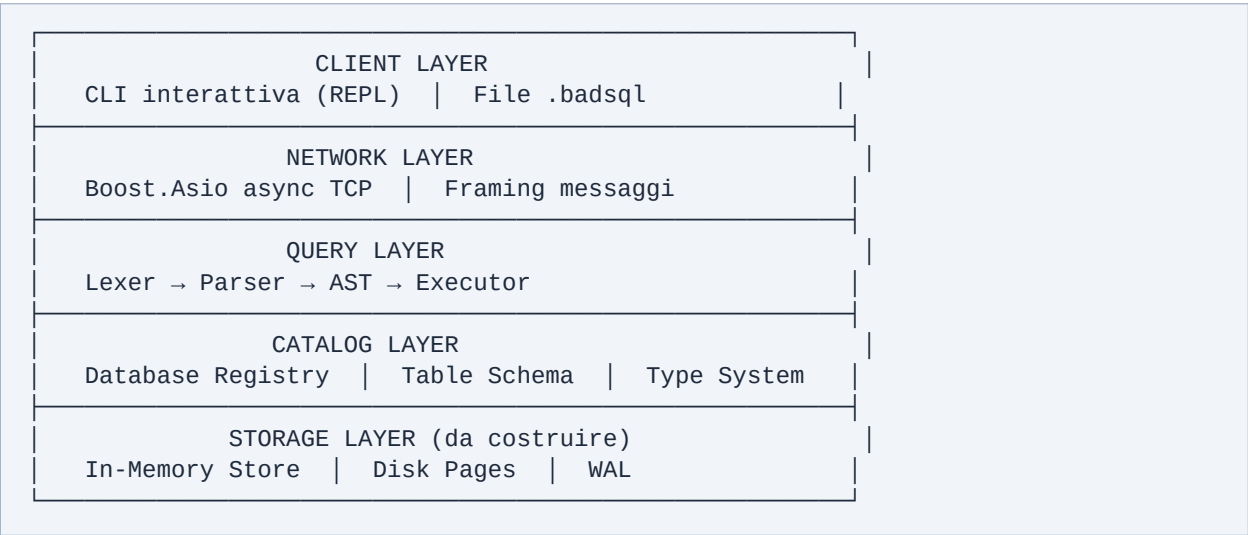
2.1 Flusso di Esecuzione Attuale

Una query percorre il seguente percorso dall'invio al risultato:



2.2 Architettura Target

Obiettivo: separare nettamente i layer e aggiungere persistenza.



3. Token System — Implementazione Reale

3.1 TokenType Enum

L'enum TokenType contiene 26 tipi di token supportati dal lexer. Si utilizza enum class (scoped enum) per evitare conflitti di nome e forzare accesso qualificato:

```
#pragma once
#include <unordered_map>
#include <string_view>
#include <optional>

namespace BadSQL {
    enum class TokenType {
        Create,
        Drop,
        Delete,
        Insert,
        Update,
        Database,
        Table,
        Use,
        From,
        Where,
        Into,
        Values,
        All,           // *
        Identifier,
        Number,
        String,
        OpenParen,
        CloseParen,
        Comma,
        SemiColon,
        I32,
        U32,
        F32,
        F64,
        Varchar,
        Char
    };
};
```

3.2 Position Struct

Ogni token mantiene la posizione nel source file (line e column) per error reporting accurato:

```
struct Position {
    size_t line;
    size_t column;
};
```

3.3 Token Struct

La struttura Token combina il tipo con la posizione nel codice sorgente:

```
struct Token {
    TokenType type;
    Position position;
};
```

3.4 Keyword Matching con unordered_map

Il lexer usa un unordered_map statico per il mapping case-sensitive di keyword string_view a TokenType. Lookup O(1) medio, nessuna copia di memoria:

```
static std::unordered_map<std::string_view, TokenType> tokenMap = {
    {"CREATE", TokenType::Create},
    {"DROP", TokenType::Drop},
    {"DELETE", TokenType::Delete},
    {"INSERT", TokenType::Insert},
    {"UPDATE", TokenType::Update},
    {"DATABASE", TokenType::Database},
    {"TABLE", TokenType::Table},
    {"USE", TokenType::Use},
    {"FROM", TokenType::From},
    {"WHERE", TokenType::Where},
    {"INTO", TokenType::Into},
    {"VALUES", TokenType::Values},
    {"i32", TokenType::I32},
    {"u32", TokenType::U32},
    {"f32", TokenType::F32},
    {"f64", TokenType::F64},
    {"varchar", TokenType::Varchar},
    {"char", TokenType::Char}
};
```

3.5 tryMatchToken Function

La funzione inline tryMatchToken usa std::optional per lookups sicuri senza eccezioni:

```
inline std::optional<TokenType> tryMatchToken(std::string_view token) {
    if (tokenMap.contains(token))
        return tokenMap.at(token);
    return std::nullopt;
}
```

4. Il Linguaggio BadSQL

4.1 Sintassi Attualmente Supportata

Le istruzioni già gestite dal lexer e che il parser sarà in grado di riconoscere:

```
-- Gestione database
CREATE DATABASE db;
USE db;

-- Definizione tabelle con tipi nativi
CREATE TABLE utenti (
  id    u32,
  eta   i32,
  nome  varchar(255),
  peso  f32
);

-- Operazioni DML
INSERT INTO utenti VALUES (1, 25, 'Alice', 65.5);
SELECT * FROM utenti;
UPDATE utenti SET eta = 26 WHERE id = 1;
DELETE FROM utenti WHERE id = 1;
DROP TABLE utenti;
```

4.2 Sistema di Tipi

Tipo BadSQL	TokenType	Descrizione	Stato
i32	I32	Intero con segno 32-bit	✓ Riconosciuto
u32	U32	Intero senza segno 32-bit	✓ Riconosciuto
f32	F32	Floating point 32-bit	✓ Riconosciuto
f64	F64	Double precision 64-bit	✓ Riconosciuto
varchar(n)	Varchar	Stringa con limite n	✓ Riconosciuto
char	Char	Carattere singolo	✓ Riconosciuto

4.3 Token Stream di Esempio

Per la query 'SELECT * FROM utenti;', il lexer produce la seguente sequenza di token:

```
Token(type=TokenType::Select, position={line=1, column=1})
Token(type=TokenType::All, position={line=1, column=8})
Token(type=TokenType::From, position={line=1, column=10})
Token(type=TokenType::Identifier, position={line=1, column=15})
Token(type=TokenType::SemiColon, position={line=1, column=21})
```

4.4 Grammatica BNF — Subset Attuale

Grammatica formale delle istruzioni riconosciute:

```
program      ::= statement* EOF

statement    ::= create_db | use_db | create_table
               | drop_table | insert_stmt | select_stmt

create_db    ::= 'CREATE' 'DATABASE' identifier ';'
use_db       ::= 'USE' identifier ';'

create_table ::= 'CREATE' 'TABLE' identifier '(' col_defs ')' ';'
col_defs     ::= col_def (',' col_def)*
col_def      ::= identifier type_name
type_name    ::= 'u32' | 'i32' | 'f32' | 'f64' | 'varchar' '(' integer ')' |
               'char'

drop_table   ::= 'DROP' 'TABLE' identifier ';'

insert_stmt  ::= 'INSERT' 'INTO' identifier 'VALUES' '(' values ')' ';'
values       ::= value (',' value)*
value        ::= number | string_literal

select_stmt  ::= 'SELECT' '*' 'FROM' identifier ';'

identifier   ::= [a-zA-Z][a-zA-Z0-9_]*
string_literal ::= '"' [^']* '"'
number       ::= [0-9]+ ('.' [0-9]+)?
```

5. Componenti Interni C++

5.1 Dettagli Implementativi Token System

5.1.1 `std::string_view` per Performance

L'implementazione usa `string_view` sia nella mappa che nel parametro della funzione di lookup. Questo evita allocazioni di memoria e permette al lexer di operare su buffer senza copie. Particolarmente importante per performance durante la tokenizzazione di file grandi.

5.1.2 `std::optional` per Error Handling

La funzione `tryMatchToken` ritorna `std::optional<TokenType>` anzichè lanciare eccezioni. Questo consente al parser di distinguere tra token riconosciuti e token sconosciuti (che diventeranno Identifier, Number o String).

5.1.3 Position Tracking per Error Reporting

Ogni token mantiene `line` e `column`. Questo permette error reporting di alta qualità: quando il parser incontra un errore, può dire esattamente dove si trovava nel file sorgente. Essenziale per debugging di query complesse.

5.1.4 Namespace `BadSQL::` per Organizzazione

Tutto il codice è incapsulato nello namespace `BadSQL::` per evitare conflitti di nomi con altre librerie e mantenere lo spazio dei nomi pulito durante l'embedding in applicazioni terze.

5.2 Parser (Pianificato)

Il parser consumerà la sequenza di Token prodotta dal lexer e costruirà un Abstract Syntax Tree (AST) tipizzato. Implementazione prevista in v0.2 con supporto per gerarchie di nodi AST e error recovery.

5.3 AST Nodes (Struttura Target)

Strutture dati pianificate per rappresentare ogni tipo di statement:

```
struct ASTNode { virtual ~ASTNode() = default; };

struct CreateDatabaseStmt : ASTNode { std::string db_name; };
struct UseDatabaseStmt : ASTNode { std::string db_name; };
struct DropTableStmt : ASTNode { std::string table_name; };

struct ColumnDef {
    std::string name;
    TokenType type;
    int size = 0;
};

struct CreateTableStmt : ASTNode {
    std::string table_name;
    std::vector<ColumnDef> columns;
};
```

5.4 Catalog e Storage In-Memory

Il catalogo è la struttura dati centrale che mantiene tutto lo stato del server in RAM durante questa fase alpha:

```
using Row = std::vector<std::string>;
using RowSet = std::vector<Row>;

struct TableSchema {
    std::string name;
    std::vector<ColumnDef> columns;
    RowSet rows;
};

struct Database {
    std::string name;
    std::unordered_map<std::string, TableSchema> tables;
};

struct Catalog {
    std::unordered_map<std::string, Database> databases;
    std::string current_db;
};
```

5.5 Executor (Pianificato)

L'executor visiterà ogni nodo AST e delegherà l'operazione al catalogo tramite pattern dispatch:

```
class Executor {
    Catalog& catalog;
public:
    std::string execute(const ASTNode& node) {
        if (auto* s = dynamic_cast<const CreateDatabaseStmt*>(&node))
            return execCreateDatabase(*s);
        if (auto* s = dynamic_cast<const CreateTableStmt*>(&node))
            return execCreateTable(*s);
        // ... altri statement
        return "ERROR: unknown statement\n";
    }
};
```

6. Struttura del Repository

6.1 Layout Attuale

```
BadSQL/
├── CMakeLists.txt      # Build: CMake 3.30, C++23, Boost
├── README.md           # Work in progress
├── test.badsql         # Script di query per test
├── .gitignore
├── src/
│   ├── token.h         # ✓ Implementato
│   └── *.cpp / *.h     # Parser, server, executor, catalog
```

6.2 Struttura Target Proposta

```
BadSQL/
├── CMakeLists.txt
├── README.md
├── test.badsql
├── tests/               # test unitari
│   ├── test_lexer.cpp
│   ├── test_parser.cpp
│   └── test_executor.cpp
├── docs/
│   └── design.md
├── src/
│   ├── main.cpp        # entry point + server loop
│   ├── token.h         # ✓ Completato
│   ├── lexer.hpp / lexer.cpp
│   ├── parser.hpp / parser.cpp
│   ├── ast.hpp         # gerarchia nodi AST
│   ├── executor.hpp / executor.cpp
│   ├── catalog.hpp     # Catalog, Database, TableSchema
│   └── server.hpp / server.cpp
```

7. Build e Utilizzo

7.1 Dipendenze

Dipendenza	Versione minima	Uso
GCC / Clang	GCC 13+ / Clang 17+	Compilatore C++23
CMake	3.30+	Build system
Boost	1.83+	ASIO per networking async

7.2 Compilazione e Avvio

```
git clone https://github.com/RickIsGone/BadSQL.git
cd BadSQL
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
./build/BadSQL          # avvia server su porta 5432
```

7.3 Invio Query

Il server accetta file .badsql come stream di testo su TCP:

```
# Con il server in esecuzione:
nc 127.0.0.1 5432 < test.badsql
```

8. Roadmap di Sviluppo

8.1 v0.2 — Parser Completo

- Costruzione AST da token stream
- WHERE clause con operatori =, !=, <, >, <=, >=
- SELECT con lista colonne specifiche
- UPDATE ... SET ... WHERE
- DELETE FROM ... WHERE
- Operatori logici AND, OR, NOT
- Error recovery nel parser

8.2 v0.3 — Executor e Risposta Strutturata

- Interpretazione AST
- Operazioni CRUD nel catalogo
- Formato risposta tabulare: header + righe + count
- Message framing per protocollo

8.3 v0.4 — Persistenza su Disco

- Formato file binario a pagine (4 KB)
- Write-Ahead Log (WAL) per durabilità
- Recovery al boot: rileggi WAL

8.4 v0.5 — Indici e Performance

- B-Tree index su PRIMARY KEY
- ORDER BY con sort in-memory
- LIMIT / OFFSET

8.5 v1.0 — Feature Complete

- Transazioni: BEGIN / COMMIT / ROLLBACK
- JOIN tra tabelle
- Funzioni aggregate: COUNT, SUM, AVG, MIN, MAX
- Multi-connection con Boost.Asio coroutines
- CLI REPL interattiva
- Test suite automatizzata

8.6 Tabella Riepilogativa

Versione	Focus	Features chiave
v0.1.0 (attuale)	Lexer + Token	TokenType enum, Position tracking, Keyword matching
v0.2.0	Parser	AST construction, WHERE, SELECT col, UPDATE, DELETE
v0.3.0	Executor	Catalogo operations, Risposta strutturata
v0.4.0	Persistenza	Disk pages, WAL, recovery
v0.5.0	Performance	B-Tree index, ORDER BY, LIMIT
v1.0.0	Complete	Transazioni, JOIN, aggregate, multi-conn

9. Decisioni Architettureali

9.1 C++23 come Linguaggio

- `std::optional` per error handling senza eccezioni
- `std::unordered_map` per $O(1)$ keyword lookup
- `std::string_view` per zero-copy string operations
- enum class per scoped enum type safety
- Massimo controllo sulle allocazioni di memoria

9.2 Porta 5432

La porta 5432 è quella di PostgreSQL. BadSQL la usa per familiarità e compatibilità futura con wire protocol PostgreSQL, senza causare conflitti se PostgreSQL non gira sulla stessa macchina durante lo sviluppo.

9.3 Storage In-Memory First

Partire con storage in-memory (`std::vector` per righe, `std::unordered_map` per tabelle) permette di concentrarsi su lexer, parser ed executor senza complessità di I/O. La persistenza verrà aggiunta come layer separato in v0.4 senza rompere l'architettura soprastante.

9.4 Token System con Position Tracking

Mantenere line e column in ogni token (non solo nel lexer) abilita error reporting accurato durante parsing e execution. Fondamentale per debugging di query complesse in ambienti embedded.

9.5 File .badsql e Architettura TCP

Il design prevede che le query arrivino al server come testo su TCP. Il file .badsql è il formato di input: può essere inviato da qualsiasi tool (netcat, CLI futura, script). Questo disaccoppia completamente il formato delle query dal meccanismo di trasporto.

10. Glossario

Termine	Definizione
AST	Abstract Syntax Tree — albero che rappresenta la struttura del codice
Catalog	Struttura in-memory che mantiene database, tabelle, schemi
DML	Data Manipulation Language — operazioni sui dati (INSERT, SELECT, UPDATE, DELETE)
DDL	Data Definition Language — istruzioni che modificano lo schema (CREATE, DROP)
enum class	Enum scoped per type safety (accesso qualificato richiesto)
Lexer	Tokenizzatore: trasforma testo raw in sequenza di Token
Parser	Analizzatore sintattico: costruisce AST da Token
Position	Line e column nel source file per error reporting
std::optional	Wrapper per valore opzionale o errore (non eccezione)
string_view	Reference a stringa senza copia di memoria
Token	Unità elementare riconosciuta dal lexer (tipo + posizione)
TokenType	Enum dei 26 tipi di token supportati
unordered_map	Hash map per O(1) keyword lookup